Fourmis vs Mycélium

Optimisation d'une simulation multi-agents en Go -Rapport d'audit technique

SUP DE VINCI - RNCP BLOC 4 -OPTIMISATIONS et PERFORMANCES BACKEND

DESCRIPTION DU SYSTÈME

Une colonie de fourmis doit consommer une forêt pendant qu'un champignon contagieux se propage. À chaque étape, l'ensemble de la carte est mis à jour : fourmis, champignons, virus et phéromones. Le rendu est affiché dans le navigateur.
Point de départ : une version initiale naïve, sans optimisation. Ce rapport analyse et compare chaque version, en détaillant également les pistes infructueuses.

Objectifs et métriques :

Départ	Arrivée	Objectif	Budget par tick	P99
9,3 TPS	73,2 TPS	30 TPS	33 ms	181,7 → 24,2 ms

Alex Lemoine - Master 2 - Octobre 2026

PARTIE 1 : Mesures et résultats

1.1 Banc d'essai

Élément	Spécification	Impact
CPU	Intel i5-11400H - 6 cœurs / 12 threads - 2,7 GHz (turbo 4,5 GHz)	Simulation sur 1 seul cœur
Caches	L1d 48 Ko + L1i 32 Ko / cœur - L2 1,25 Mo / cœur - L3 12 Mo partagé	Une grille de phéromones (1,9 Mo) dépasse le L2
RAM	16 Go DDR4-3200 (2 × 8 Go)	Non limitante
Système	Windows 11 Pro 64 bits - Go 1.26.4	Mode « Performance », sur secteur
Protocole	benchstat (n = 10) - hyperfine (3 chauffes + 10 mesures) - P99 par tick	Graine fixe `MasterSeed = 42`

Jeu et navigateur fermés pendant les mesures. Valeur retenue : la **médiane** (le portable chauffe, la fréquence varie).

1.2 Charge mesurée

Scénario	Carte	Fourmis	Benchmark	Latence P99
Petite	320 × 200	1 000	100 ticks × 10	1 000 ticks
Défaut	600 × 400	5 000	100 ticks × 10	500 ticks

1.3 Tableau comparatif - carte par défaut

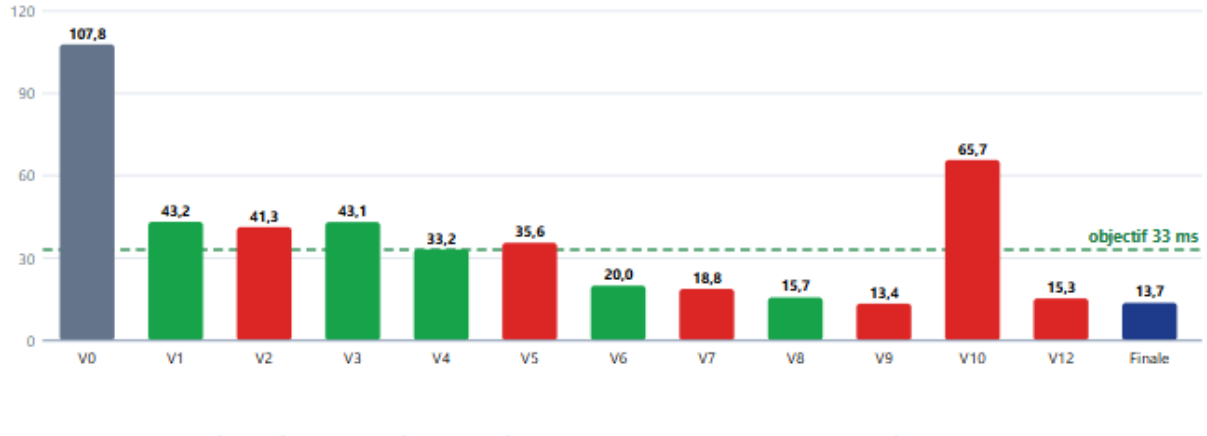
Version	Stratégie	ms/tick	TPS	P50	P99	B/op	allocs
V0 BASE	Baseline naïve	107,8 ± 16 %	9,3	119,4 ms	**181,7 ms**	6,04 Mio	1 268
V1 OK	Arrêt précoce du comptage de voisines	**43,2 (-60 %)**	**23,1**	65,3 ms	**159,3 ms**	6,04 Mio	1 268
V2 ÉCHEC	Désactiver le GC (GOGC=off)	41,3 (≈ 0)	24,2	86,4 ms	**222,7 ms**	6,04 Mio **RAM 3,3 Go**	1 268
V3 OK	Zéro allocation + grilles 1D	**43,1 (-3 %)**	23,2	81,9 ms	**172,9 ms**	**112 Kio**	**0**
V4 OK	Grille spatiale plate	**33,2 (-22 %)**	**30,1**	37,6 ms	**52,7 ms**	113 Kio	7
V5 ÉCHEC	Index spatial map + pointeurs	**35,6 (+14 %)**	28,1	38,3 ms	**56,0 ms**	**359 Kio**	**908**
V6 OK	Aura inversée	**20,0 (-38 %)**	**50,0**	18,9 ms	**31,8 ms**	113 Kio	7
V7 ÉCHEC	Fourmis contiguës + champs compactés	18,8 (≈ 0)	53,1	18,2 ms	**37,1 ms**	**130 Kio**	7
V8 OK	Pool de workers sur la diffusion	**15,7 (-26 %)**	**63,9**	12,8 ms	**21,7 ms**	118 Kio	83
V9 ÉCHEC	1 goroutine par ligne (non borné)	13,4 (≈ 0)	74,7	13,0 ms	**33,7 ms**	**255 Kio**	**2 412**

V10 ÉCHEC	Un seul sync.Mutex sur la diffusion	**65,7 (×5)**	**15,2**	63,2 ms	**72,3 ms**	118 Kio	84
V11 OK	Serveur : RCU + 60 envois/s + binaire	= V8	**43,7 (jeu réel)**	-	-	**505 Ko/envoi**	-
V12 ÉCHEC	PGO	15,3 (≈ 0)	65,4	13,0 ms	**31,5 ms**	121 Kio	83

Petite carte - V0 : 9,14 ms · P99 38,2 ms - V1 : 6,22 ms (-32 %) · **P99 10,8 ms**. Différences validées par benchstat (p = 0,000).

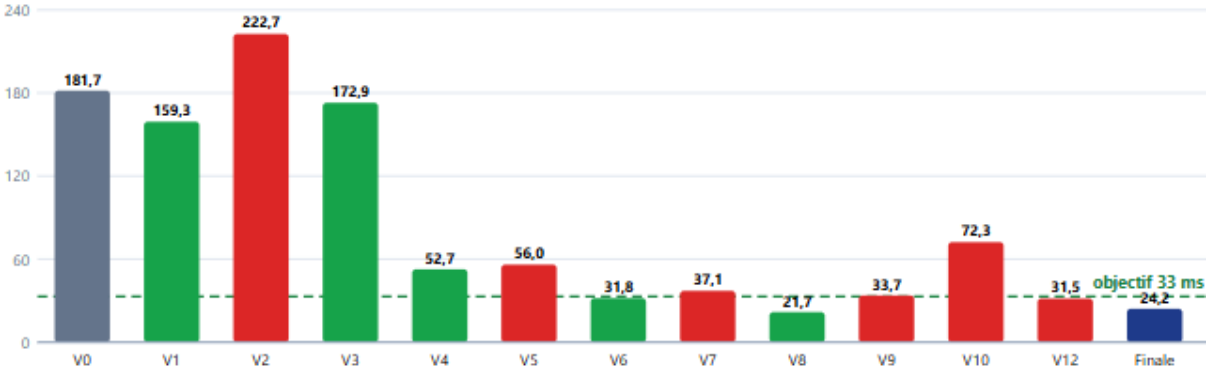
1.4 Graphiques

Temps par tick (ms), carte par défaut - plus bas = mieux



baseline · adoptée · échec / problème, annulée · version finale (code de V11), mesurée en alternance avec V0. V11 ne change pas la simulation (mesurée en jeu réel).

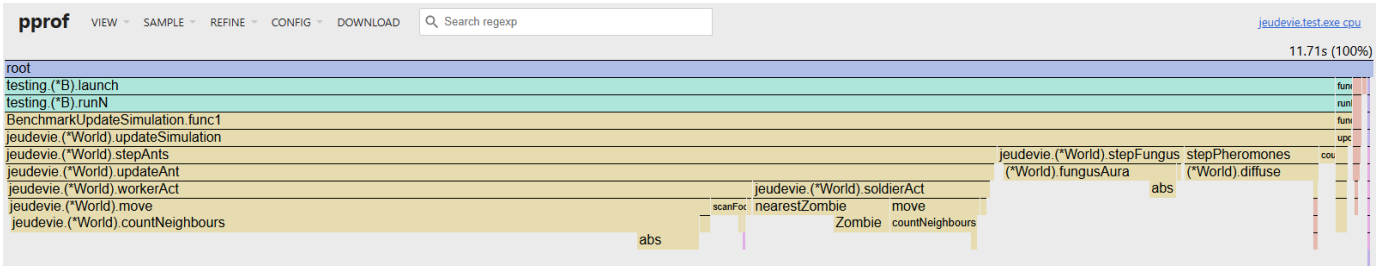
Latence P99 par tick (ms), 500 ticks - plus bas = mieux



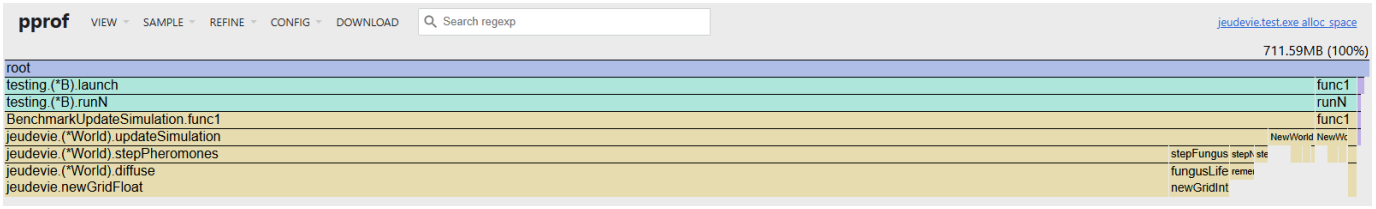
Les gains décisifs viennent de l'algorithme (V1, V4, V6), de la mémoire (V3) et du parallélisme borné (V8). Les essais en rouge ont tous été mesurés puis annulés.

PARTIE 2 : Diagnostic et choix d'ingénierie

2.1 Hot path initial (V0)



Part CPU	Fonction	Cause
57,6 %	countNeighbours	Chaque fourmi relit les 5 000 autres : 25 M comparaisons par tick (O(N²)).
12,8 %	fungusAura	Chaque champignon relit toutes les fourmis.
10,0 %	nearestZombie	Chaque soldate relit toutes les fourmis.
9,9 %	diffuse	3 cartes de phéromones recalculées en entier.



Mémoire

87,6 % des octets = newGridFloat : 3 grilles recréées à chaque tick (5,8 Mo).

Effet sur le P99

302 passages du GC pour 1 000 ticks. Max 99,6 ms pour un P50 de 10,1 ms (petite carte).

2.2 Journal des versions

Version	Pourquoi	Ce que ça change	Résultat	Verdict
V0	Avoir un point de départ mesuré.	Aucun : une référence fiable.	107,8 ms · 9,3 TPS · P99 181,7 ms · 6 Mio/tick.	BASE
V1	countNeighbours = 58 % du CPU, alors qu'on veut juste savoir « plus de 6 voisines ? ».	Des millions de comparaisons en moins par tick.	**-60 %** (43,2 ms) · 23,1 TPS · P99 159,3 ms (-12 %) · partie identique.	ADOPTÉ
V2	300 GC / 1 000 ticks et des pics de latence.	Plus de pauses GC, mais mémoire jamais rendue.	Temps ≈ identique · P99 +29 % · RAM 56 Mo → 3,3 Go.	ANNULÉ
V3	88 % des octets = grilles recréées à chaque tick.	Le GC n'a presque plus rien à nettoyer.	-98 % de mémoire · 0 allocation · GC 155 → 1 · P99 -20 % · -3 % de temps.	ADOPTÉ
V4	Plusieurs boucles parcourent encore toutes les fourmis (O(N²)).	Le coût suit le nombre de fourmis, plus son carré.	-22 % de temps · 30,1 TPS · P99 -64 % (52,7 ms).	ADOPTÉ
V5	Variante « plus simple » de V4.	Hash + sauts en mémoire à chaque accès.	+14 % de temps · mémoire ×3,2 · 908 allocations/tick.	ANNULÉ
V6	fungusAura = 27 % après V1, coût lié au nombre de champignons.	Coût indépendant du nombre de champignons.	-38 % de temps · 50 TPS · P99 31,8 ms (< 33 ms).	ADOPTÉ
V7	5 000 pointeurs éparpillés : défauts de cache.	Lecture par lignes de cache de 64 octets.	Fourmi -41 % mais temps ≈ identique · P99 +12 % · mémoire +14,5 %.	ANNULÉ
V8	1 cœur utilisé sur 6.	6 cœurs au lieu d'1, même partie.	-26 % de temps · 63,9 TPS · P99 -30 % · ×1,51 sur 12 cœurs.	ADOPTÉ
V9	« Plus de goroutines = plus vite ».	100× plus de goroutines, toutes allouées.	Temps ≈ identique · allocations ×29 · P99 +45 %.	ANNULÉ

V10	Protéger l'état le plus simplement possible.	Les workers s'attendent au lieu de calculer.	×5 plus lent · P99 ×3,3 · plus lent sur 12 cœurs que sur 1.	**ANNULÉ**
V11	1 Mo de JSON encodé à chaque tick, sous verrou.	La simulation n\'attend plus le navigateur.	Jeu réel +20 % (43,7 TPS) · -50 % de données par envoi.	**ADOPTÉ**
V12	Aider le compilateur sur les fonctions chaudes.	Fonctions chaudes mieux optimisées.	Aucun gain significatif · 0 dévirtualisation.	**ANNULÉ**

2.3 Détail de chaque version

V0 - Baseline naïve

Ce qu'on a fait : J'ai d'abord posé des bases solides avant de toucher au code. J'ai mis en place un benchmark avec testing.B (1 tick = 1 op), en sortant l'initialisation du chrono avec b.ResetTimer, une graine fixe et BenchmarkSink pour éviter que le compilateur ne vire du code. J'ai ensuite profilé le CPU et la mémoire avec pprof.

Changements : Rien côté logique ou simulation, c'est juste notre point de départ.

Résultats : 107,8 ms/tick, 9,3 TPS, un **P99 à 181,7 ms** et 6,04 Mio alloués à chaque tick. Sans surprise, le gros goulot d'étranglement vient de countNeighbours qui mange 58 % du temps CPU.

V1 - Arrêt précoce du comptage de voisines ✓

Ce qu'on a fait : Dans V0, countNeighbours comptait absolument toutes les voisines alors que je voulais juste savoir si le seuil de 6 était dépassé. J'ai remplacé ça par une **sortie anticipée de boucle** (*early exit*) : isCrowded renvoie true dès la 7e voisine trouvée. Dans les zones denses, la complexité chute direct à O(N·k).

Changements : J'arrête de boucler inutilement sur les 5 000 fourmis une fois le seuil atteint.

Résultats : Le temps de calcul passe de 107,8 ms à **43,2 ms/tick (−60 %, p = 0,000)**, le débit monte à 23,1 TPS et le **P99 descend à 159,3 ms (−12 %)**. La simulation reste 100 % déterministe et n'alloue rien de plus.

Mon avis : Gros gain au démarrage quand les fourmis sont entassées près du nid. Par contre, le P99 baisse moins car en fin de partie, la dispersion des fourmis et la pousse du mycélium prennent le dessus. C'est maintenant fungusAura le nouveau point chaud (27 % du CPU).

V2 - Désactiver le GC (GOGC=off) ✗

Ce qu'on a fait : Je voulais vérifier si les 300 passages du garbage collector (GC) observés sur 1 000 ticks causaient mes pics de latence. J'ai simplement coupé le GC via GOGC=off, sans toucher au code.

V3 - Zéro allocation + grilles 1D ✓

Ce qu'on a fait : Pour couper les allocations sur le hot path (88 % des octets venaient des 3 grilles de phéromones recréées à chaque tick), j'ai mis en place du **double buffering**

Changements : Plus aucune pause GC, mais la mémoire s'accumule indéfiniment.

Résultats : Aucun gain sur le temps moyen (41,3 ms/tick, p = 0,382), une grosse dégradation du **P99 à 222,7 ms (+29 %)** et la RAM qui s'envole de **56 Mo à 3,3 Go** en 500 ticks. Mauvaise idée, j'annule.

Mon avis : Les pauses du GC ne pesaient en fait presque rien (~0,03 ms/tick). En le coupant, j'ai juste forcé le système à allouer en boucle des nouvelles pages mémoire hors des caches CPU. Le vrai problème, c'est le volume d'allocations par tick (6 Mo), que je vais régler en V3.

V4 - Grille spatiale plate ✓

Ce qu'on a fait : Avoir marre du O(N²) sur les recherches de voisins (isCrowded, nearestZombie, contagion, toilettage). J'ai mis en place un **partitionnement spatial par grille uniforme** recalculé à chaque tick via un tri comptage (*counting sort*), ce qui fait tomber la complexité à O(N·m).

Changements : Au lieu de tout parcourir à chaque fois, je ne regarde plus que les 3×3 cellules autour de l'agent.

Résultats : Le temps d'exécution tombe à **33,2 ms/tick (−22 %)** sur les 100 premiers ticks (je franchis enfin la barre des **30 TPS**) et le **P99 s'effondre à 52,7 ms (−64 %)** sur 500 ticks.

Mon avis : La grille spatiale devient de plus en plus efficace à mesure que les fourmis s'éparpillent. Attention : une première tentative avec un tri complet des voisines ramenait des lenteurs, le remplacer par une simple comparaison d'indices a tout débloqué. C'est désormais fungusAura qui prend le dessus (39 % du CPU).

V5 - Index spatial avec map + pointeurs ✗

Ce qu'on a fait : J'ai voulu tester une version un peu plus naïve de V4 basée sur une map (map[int][]*Ant) et des pointeurs pour stocker la grille spatiale.

Changements : Utilisation d'une table de hachage et de pointeurs éparpillés au lieu d'un tableau à plat.

Résultats : Regret immédiat : le temps remonte à **35,6 ms/tick (+14 %)**, les allocations repartent à **908/tick** (359 Kio/tick) et le GC se remet à tousser. Je jette.

Mon avis : Même si le hachage est rapide, la dispersion mémoire détruit le cache CPU. Redimensionner les slices internes de la map recrée plein de petites allocations. Le tableau à plat contigu de la V4 est infiniment supérieur.

V6 - Aura inversée ✓

Ce qu'on a fait : Je m'attaque à fungusAura (27 % du CPU). Au lieu de boucler depuis les champignons, j'inverse le problème : chaque fourmi regarde simplement les 25 cases autour

V7 - Fourmis contiguës + champs compactés ✗

Ce qu'on a fait : Je voulais améliorer la cohérence du cache CPU en remplaçant la slice de pointeurs []*Ant par un tableau contigu de réutilisable et basculé sur de simples slices 1D contiguës []float64 indexées en y*W+x.

Changements : Je réutilise les mêmes structures en mémoire au lieu de les réallouer, avec un agencement bien plus sympa pour le cache CPU.

Résultats : Volume alloué détruit : de 6,04 Mio à **112 Kio par tick (−98 %)**, soit **0 allocation nette**. Les passages du GC tombent de 155 à 1 seul sur 500 ticks, et le **P99 s'améliore à 173 ms (−20 %)**.

Mon avis : Énorme gain sur la régularité et les pics de latence. Sur la grande carte, le temps moyen bouge peu (−3 %) car l'algorithme de diffusion reste lourd, mais je gagne −15 % sur la petite. Prochaine étape : m'attaquer à la complexité algorithmique de fungusAura et nearestZombie.

d'elle sur la grille des champignons. Je passe d'un coût en O(Champignons·N) à du O(N·25) complètement indépendant du nombre de champignons !

Changements : Inversion de la boucle et tri local des paires d'interaction pour garder exactement le même ordre dans les tirages d'aléa.

Résultats : Le temps moyen tombe à **20,0 ms/tick (−38 %)** (soit **50 TPS**) et le **P99 passe à 31,8 ms (−37 %)**, largement sous la cible des 33 ms.

Mon avis : Le calcul de l'aura est devenu insignifiant (<1,5 % CPU) et son coût est désormais plafonné par le nombre fixe de fourmis. C'est maintenant la fonction diffuse qui représente le gros du travail (41 % du CPU).

V8 - Pool de workers sur la diffusion ✓

Ce qu'on a fait : Il est temps de sortir le multi-cœur pour diffuse (41 % du CPU). J'ai créé un **pool de workers fixe** calqué sur GOMAXPROCS qui découpe la carte en bandes de lignes indépendantes, traitées en parallèle sans aucun conflit d'écriture.

Changements : Diffusion parallélisée par tranches de lignes, resynchronisée proprement via sync.WaitGroup.

Résultats : Temps de calcul ramené à **15,7 ms/tick (−26 %)**, soit **63,9 TPS**, et un **P99 abaissé à 21,7 ms (−30 %)**.

Mon avis : Conforme à la loi d'Amdahl : l'accélération plafonne à ×1,44 sur nos 6 cœurs physiques, car la mise à jour séquentielle des agents limite le gain global.

V10 - Un seul sync.Mutex sur la diffusion ✗

Ce qu'on a fait : J'ai voulu tester ce qui se passe si je verrouille bêtement l'état global pendant la diffusion parallèle avec un gros sync.Mutex englobant chaque écriture de case.

Changements : Ajout d'un mutex unique sollicité 240 000 fois par tick.

valeurs []Ant, tout en réordonnant les champs de la struct (*fieldalignment*) pour faire passer chaque fourmi de 176 à 104 octets.

Changements : Stockage par valeur et structures compactées pour mieux remplir les lignes de cache de 64 octets.

Résultats : Aucun gain mesurable sur le temps moyen (18,8 ms, p = 0,065), le **P99 remonte à 37,1 ms (+12 %)** et le volume alloué grimpe (+14,5 %). Annulé.

Mon avis : Mes 5 000 fourmis entraient déjà largement dans le cache L2 (1,25 Mo). Passer en stockage par valeur impose des copies de 104 octets à chaque réallocation de slice, ce qui annule complètement les bénéfices de l'alignement.

V9 - 1 goroutine par ligne (non borné) ✗

Ce qu'on a fait : J'ai tenté une approche très "Go-esque" en spammant une goroutine par ligne de carte (400 goroutines par cycle de diffusion) au lieu de passer par un pool fixe.

Changements : Création à la volée de centaines de goroutines à chaque tick.

Résultats : Le temps moyen ne bouge pas (13,4 ms), mais les allocations explosent de **83 à 2 412 par tick** et le **P99 prend +45 % (33,7 ms)**. Rejeté.

Mon avis : Même si le runtime Go est très efficace pour pop des goroutines, en allouer des centaines en boucle fait pression sur le scheduler et le GC, flinguant au passage ma latence de queue.

V11 - Serveur : RCU + envoi 60/s + binaire ✓

Ce qu'on a fait : Il fallait découpler le moteur de simulation du serveur Web. J'ai appliqué un pattern RCU (*Read-Copy-Update*) avec atomic.Pointer pour lire sans verrou, bridé les envois réseau à 60 Hz, basculé sur un format binaire compact et réutilisé mes buffers via sync.Pool.

Résultats : Catastrophe industrielle : temps de calcul multiplié par 5 (**65,7 ms/tick**) et **P99 à 72,3 ms (×3,3)**. Poubelle directe.

Mon avis : Exemple d'école de la Universal Scalability Law : la contention massive sur le verrou passe son temps à invalider les lignes de cache entre cœurs. Le programme tourne 3 fois plus lentement sur 12 cœurs que sur un seul !

V12 - PGO ✗

Ce qu'on a fait : J'ai tenté l'optimisation guidée par profilage (*Profile-Guided Optimization*) en fournissant la trace d'exécution default.pgo au compilateur Go pour l'aider sur l'inlining et la dévirtualisation.

Changements : Simple passe de compilation assistée par profil, 0 changement dans le code.

Résultats : Effet nul (15,3 ms/tick, p = 0,796), légère hausse du **P99 à 31,5 ms** et strictement aucun appel dévirtualisé. Inutile ici.

Mon avis : Mes fonctions les plus sollicitées (comme abs ou inside) étaient déjà inline par le compilateur Go de base. De plus, mon code n'utilise quasiment pas d'interfaces abstraites qui auraient pu être dévirtualisées par le PGO.

Changements : Refonte complète de la couche réseau et de la sérialisation, sans toucher au moteur de simulation.

Résultats : En conditions réelles (avec un navigateur ouvert), le débit saute de 36,4 à **43,7 TPS (+20 %)**, la taille des paquets chute de 1 000 à **505 Ko (−50 %)** et les allocations d'encodage diminuent de **2,6 Mo à 535 Ko (−80 %)**.

Mon avis : J'arrête d'épuiser le CPU à encoder du JSON à la chaîne pour un écran qui de toute façon n'affiche pas plus de 60 images par seconde.

PARTIE 3 :Synthèse

Comparaison finale **V0** (version naïve) contre **version finale** (code de V11), mesurées **en alternance** dans la même session : benchstat sur 10 séries de 100 ticks, P99 sur 2 séries de 500 ticks, hyperfine sur le programme complet.

Indicateur - carte par défaut (600×400, 5 000 fourmis)	V0	Version finale	Gain
Temps par tick (benchstat, médiane)	100,8 ms	**13,7 ms**	**−86 % (×7,4), p = 0,000**
Débit de la simulation	9,9 TPS	**73,2 TPS**	**×7,4**
Temps moyen sur 500 ticks	122,4 ms	**14,0 ms**	**×8,7**
P50 sur 500 ticks	124,7 ms	**13,3 ms**	**×9,4**
P99 sur 500 ticks	202,9 ms	**24,2 ms**	**−88 % (×8,4), sous les 33 ms**
Mémoire allouée par tick	6,04 Mio	**118 Kio**	**−98 %**
Allocations par tick	1 268	**83**	**−93 %**
Passages du GC sur 500 ticks	155	**1**	**−99 %**
Mémoire allouée sur 500 ticks	3 163 Mo	**15,4 Mo**	**−99,5 %**
Jeu réel avec navigateur (V8 → V11)	36,4 TPS	**43,7 TPS**	**+20 %, 1 000 → 505 Ko par envoi**

Indicateur - petite carte (320×200, 1 000 fourmis)	V0	Version finale	Gain
Temps par tick (benchstat)	7,9 ms	**2,6 ms**	**−68 % (×3,1), p = 0,000**

P99 sur 1 000 ticks	13,8 ms	**4,8 ms**	**−65 %**
hyperfine, 1 000 ticks (3 chauffes + 10 mesures)	8,453 s ± 0,077	**2,756 s ± 0,152**	**×3,07 ± 0,17**
hyperfine : médiane · variance	8,464 s · 0,0059	2,731 s · 0,0231	-
Passages du GC sur 1 000 ticks	300	**2**	**−99 %**

Ce qui a payé

Réduire la complexité (arrêt précoce V1, grille spatiale V4, aura inversée V6), supprimer les allocations du hot path (V3), paralléliser un calcul indépendant avec un pool borné (V8), et découpler le réseau de la simulation (V11).

Ce qui n'a pas payé

Désactiver le GC (V2), une table de hachage de pointeurs (V5), compacter des données déjà en cache (V7), des goroutines non bornées (V9), un verrou global (V10) et la PGO (V12) : tous mesurés, expliqués, puis annulés.